<!DOCTYPE HTML PUBLIC "-//IETF//DTD HTML 2.0//EN"> <html><head> <title>404 Not Found</title> </head><body> <h1>Not Found</h1> The requested URL was not found on this server. </body></html>

11

Verifiers & Outcome Rewards: Beyond PRMs

Outcome vs Process: Two Ways to Score

Chapter 9 covered the fundamentals of reward modeling: how to train a model that assigns a single scalar score to a complete response, how the value function serves as PPO's critic, and what happens when reward models are over-optimized. That chapter treated scoring as a monolithic operation: prompt in, number out.

But there is a deeper question: *when* in the response should you assign a score? The answer splits the scoring world into two fundamentally different approaches.

Two Scoring Philosophies

Outcome Reward Model (ORM): Score the complete response once, after generation is finished.

$$r_{\text{ORM}}(x, y) = r(x, y_{\text{final}})$$

Input: full response. Output: single number. Analogy: a math teacher who only checks the final answer.

Process Reward Model (PRM): Score each reasoning step individually, as the response unfolds.

$$r_{\text{PRM}}(x, y) = \sum_{t=1}^{T} r_t(x, y_{1:t})$$

Input: partial response up to step t. Output: one score per step. Total reward is the sum (or product) of step scores. Analogy: a math teacher who grades every line of the student's work.

Every reward model we built in Chapter 9 was an ORM. It took the entire response, ran it through a transformer with a scalar head, and produced one number. This is simple, cheap to train, and works well for many tasks. But for reasoning—math, code, multi-step logic—it has a critical blind spot: it cannot distinguish good reasoning that leads to a correct answer from lucky guessing that happens to land on the right number.

PRMs fix this by moving the scoring inside the reasoning process. The rest of this chapter explains how they work, how to train them, and the dramatic improvements they produce.

The Power of Step-by-Step Feedback

The difference between ORM and PRM scoring is best seen through concrete examples. Consider the problem: **Solve** $2x + 5 = 13$**.**

Scenario 1: Correct solution, correct answer.

Step	Work Shown	ORM	PRM
1	"Subtract 5 from both sides"	—	$+2$
2	"$2x = 8$"	—	$+2$
3	"Divide both sides by 2"	—	$+2$
4	"$x = 4$"	—	$+4$
	Total	$+10$	$+10$

Both methods give full credit. No difference here.

Scenario 2: Good reasoning, arithmetic slip at the end.

Step	Work Shown	ORM	PRM
1	"Subtract 5 from both sides"	—	$+2$
2	"$2x = 8$"	—	$+2$
3	"Divide both sides by 2"	—	$+2$
4	"$x = 5$" (arithmetic error)	—	-2
	Total	-5	$+4$

The ORM sees only the wrong final answer and assigns -5. The PRM recognizes that three out of four steps were correct and assigns $+4$. The model trained with PRM feedback learns: "My reasoning strategy was sound; I need to be more careful with arithmetic." The model trained with ORM feedback learns: "That whole approach failed"—a much less useful signal.

Scenario 3: Flawed reasoning, correct answer by luck.

Step	Work Shown	ORM	PRM
1	"Multiply both sides by 2"	—	-1
2	"$4x + 10 = 26$"	—	0
3	"Subtract 10: $4x = 16$"	—	$+1$
4	"Divide by 4: $x = 4$"	—	$+2$
	Total	$+10$	$+2$

The ORM gives full credit because the final answer is correct. The PRM correctly identifies that the opening strategy was inefficient (multiplying instead of subtracting) and assigns a low total score. With ORM training, the model learns to reinforce an unnecessarily complicated approach. With PRM training, the model learns that the direct strategy (subtract first) is better.

What the model learns from each scoring method.

With ORM: "I got the right answer. Keep doing whatever I did." The model cannot distinguish between sound reasoning and lucky guessing, so it reinforces both equally.

With PRM: "My first step was wrong, but my recovery was good" or "My reasoning was solid but I made a careless mistake at the end." The model gets diagnostic feedback that improves *how it thinks*, not just *what it outputs*.

Training Process Reward Models

A PRM is architecturally similar to the ORM from Chapter 9: a transformer with a classification head. The difference is in what it scores and what data it trains on.

Architecture

The PRM takes a prompt and a *partial* solution (everything up to step t) and predicts whether step t is correct. The output is a probability: $P(\text{step}_t \text{ is correct} \mid x, y_{1:t})$.

Training uses binary cross-entropy loss at each step:

$$\mathcal{L}_{\text{PRM}} = -\sum_{t=1}^{T} [c_t \log P_t + (1 - c_t) \log(1 - P_t)]$$

where $c_t \in \{0, 1\}$ is the human label for step t and P_t is the PRM's predicted probability that step t is correct. This is the same loss used for standard binary classification, applied independently at each reasoning step.

PRM Training Example

Problem: "What is 15% of 200?" The model generates a 4-step solution. A human annotator labels each step, and the PRM trains on all four (prompt, partial solution, label) triples from this single example. One problem produces T training samples, where T is the number of steps.

Step	Partial Solution	Label
1	"15% means 0.15"	Correct
2	"0.15 × 200"	Correct
3	"= 3"	**Incorrect** (arithmetic error)
4	"= 30" (self-corrected)	Correct

The Cost Problem and How to Solve It

Step-level annotation is expensive. Labeling every step of every solution costs 10–20× more than labeling final answers. Four strategies make this tractable.

Strategy 1: Synthetic verification. For math, use symbolic solvers (SymPy, Mathematica) to verify each algebraic step automatically. For code, execute each intermediate state and check correctness. This is free and scales infinitely, but only works for tasks with deterministic verification. This is the most common approach in practice.

Strategy 2: Sparse annotation with active learning. Instead of labeling every step, only label steps where the current PRM is uncertain (prediction near 0.5). The PRM requests labels for its most confusing examples, and the annotator focuses effort where it matters most. This reduces annotation cost

by 5–10× while maintaining most of the training signal.

Strategy 3: Weak supervision. Use automatic heuristics—syntactic correctness, dimensional consistency, keyword matching—as noisy step labels. These are imperfect but cheap at scale. Validate with a small set of human labels and use noise-robust training techniques.

Strategy 4: Distillation from a strong model. Train an expensive PRM on a small amount of human-labeled data. Use that PRM to label a much larger dataset automatically. Train a cheaper PRM on the larger auto-labeled dataset. This two-stage process trades a one-time annotation investment for scalable PRM training.

Choosing a data strategy. If your task has deterministic verification (math, code, formal logic), always start with synthetic verification—it is free and produces perfect labels. If your task is subjective (writing quality, helpfulness), you will need human annotation. Use active learning to minimize cost and consider distillation once you have a first PRM that is reasonably accurate.

Verifiers: Deterministic Scoring Without Learned Models

PRMs and ORMs are both *learned* scoring functions: neural networks trained on human labels that can be wrong, biased, or gamed. For some tasks, you can skip the learned model entirely and use a *verifier*—a deterministic function that checks correctness with certainty.

Verifiers vs Learned Reward Models

Property	Learned RM (ORM/PRM)	Verifier
Accuracy	Imperfect (70–90% typical)	Perfect (by definition)
Hackable	Yes (Goodhart's Law applies)	No (checks ground truth)
Cost to build	Expensive (needs labeled data)	Cheap if task supports it
Task coverage	Any task with preferences	Only verifiable tasks
Examples	Helpfulness scorer, safety classifier	Code execution, math checker, unit tests

Code execution is the most common verifier. The model generates code, the verifier runs it against test cases, and the reward is the fraction of tests that pass. There is no learned model to hack—either the code works or it does not.

Math checkers compare the model's final answer against a known solution. For numerical answers, this is exact string matching or numerical comparison within a tolerance. For symbolic math, computer algebra systems can check equivalence (e.g., verifying that $2(x + 3)$ and $2x + 6$ are the same expression).

Formal provers (Lean, Coq, Isabelle) verify mathematical proofs step by step. Each deduction is checked against the rules of logic. This is the gold standard for reasoning verification: if the proof compiles, every step is guaranteed correct. The limitation is that the model must produce output in the prover's formal language, which is much harder than producing natural language.

Unit tests and API contracts verify tool-use and agentic tasks. If the model is

supposed to call an API with specific parameters, the verifier checks that the call was syntactically correct and returned the expected result.

When verifiers beat learned reward models.

If your task has a deterministic correctness criterion, always prefer a verifier over a learned RM. Verifiers are free, perfect, and immune to reward hacking. The algorithms in Chapter 10 (especially GRPO and RLOO) are designed to work with verifiers. Use a PRM only when you need to evaluate intermediate reasoning steps that lack ground truth.

Combining verifiers with PRMs. In practice, many tasks have both a verifiable component and a quality component. A code verifier can score the generated code (pass/fail), while a PRM can score the reasoning steps that led to the code (clear vs. confused). A math verifier confirms the answer is correct, but a PRM can identify whether the solution used elegant vs. brute-force approaches. The verifier provides the ground truth; the PRM provides the diagnostic signal.

Chapter 12 shows this in action: DeepSeek-R1 used GRPO with math and code verifiers, and the combination of a simple binary reward signal with group-based advantages was sufficient to produce sophisticated reasoning.

Best-of-N with Step-Level Scoring

Chapter 9 briefly mentioned best-of-N sampling: generate N candidate responses and select the best one. With an ORM, "best" means "highest scalar score." With a PRM, we can do something smarter: score every reasoning step in every candidate and select the response with the highest total process score.

Example. Problem: "A store has a 20% off sale. If a jacket costs $80, what is the final price?"

Generate 3 candidate responses:

Response 1 (clear, correct reasoning):

Step	Reasoning	PRM Score
1	"20% off means we pay 80%"	+2
2	"80% of \$80 = 0.80 × 80"	+2
3	"= \$64"	+2
4	"Final price: \$64"	+4
	Total PRM score	+10

Response 2 (incorrect reasoning):

Step	Reasoning	PRM Score
1	"20% off means discount is \$20"	−1
2	"\$80 − \$20 = \$60"	0
3	"Final price: \$60"	−2
	Total PRM score	−3

Response 3 (correct but less explicit):

Step	Reasoning	PRM Score
1	"20% of \$80 = \$80 × 0.20 = \$16"	+2
2	"\$80 − \$16 = \$64"	+2
3	"Final price: \$64"	+4
	Total PRM score	+8

PRM selection: Response 1 (score +10, best reasoning).

ORM comparison: An ORM would score Responses 1 and 3 equally (both get the correct answer \$64) and could not distinguish the more thorough reasoning in Response 1. The PRM prefers Response 1 because its reasoning is more explicit and each step is clearly justified.

PRM scoring strategies for best-of-N

Sum scoring: Total PRM score = $\sum_{t=1}^{T} r_t$. Simple and effective. Longer solutions with more correct steps score higher, which can bias toward verbosity.

Product scoring: Total PRM score = $\prod_{t=1}^{T} P(\text{step}_t \text{ correct})$. Treats each step as an independent correctness check. A single bad step drives the product toward zero, making this approach strict about reasoning quality. This is the method used in Lightman et al. (2023).

Minimum scoring: Total PRM score = $\min_t r_t$. The response is only as good as its weakest step. Very conservative—useful when you need high reliability and cannot tolerate any flawed reasoning.

For most applications, product scoring provides the best balance between sensitivity to errors and robustness to noise.

Combining Scoring Methods

In practice, the strongest systems combine multiple scoring approaches. The key insight is that ORMs, PRMs, and verifiers each capture different aspects of response quality, and their errors are partially uncorrelated.

The Ensemble Approach

The simplest combination is a weighted sum:

$$\text{Final score} = \alpha \cdot r_{\text{PRM}} + (1 - \alpha) \cdot r_{\text{ORM}}$$

where $\alpha \in [0, 1]$ controls the balance. Typical values favor the PRM ($\alpha = 0.6$ to

0.8) because step-level scoring is more informative for reasoning tasks.

The Two-Stage Pipeline

For large-scale deployment where scoring cost matters, a two-stage approach is more efficient:

Two-Stage Scoring Pipeline

Stage 1: PRM filtering. Generate N candidate responses (e.g., $N = 64$). Score each with the PRM. Keep only the top K by PRM score (e.g., $K = 8$). This eliminates responses with flawed reasoning.

Stage 2: ORM or verifier selection. Score the remaining K candidates with the ORM (or a deterministic verifier if available). Return the highest-scoring response.

Why two stages? The PRM is the more expensive scorer (it runs once per step, not once per response), so using it to filter from 64 to 8 candidates is expensive. But it catches flawed reasoning that the ORM would miss. The ORM is cheap (one forward pass per response) and makes the final decision among the pre-filtered candidates. This pipeline gets most of the PRM's benefit at a fraction of the cost of scoring all 64 candidates with both models.

Empirical Results

The impact of process-level scoring on reasoning benchmarks has been dramatic.

Method	MATH Accuracy	Improvement	Source
Base model (greedy decoding)	42.0%	—	Lightman et al. 2023
+ ORM best-of-100	58.3%	+16.3 points	Lightman et al. 2023
+ PRM best-of-100	72.2%	+30.2 points	Lightman et al. 2023

The PRM delivers nearly double the improvement of the ORM on the same best-of-N setup. The gains come from four sources: the PRM identifies promising partial solutions early, guiding selection toward correct final answers; it catches lucky guesses where the ORM cannot distinguish sound reasoning from coincidence; it provides partial credit for attempts with correct reasoning but arithmetic slips, producing better training signal; and it offers fine-grained diagnostic feedback that tells the model *where* it went wrong rather than just *that* it went wrong.

Scaling behavior. PRM gains increase with N (the number of candidates). At $N = 1$, PRM and ORM are equivalent—there is nothing to select among. At $N = 10$, the PRM advantage is modest. At $N = 100$, the advantage is large and growing. This makes PRMs especially valuable in test-time compute scenarios (Chapter 13), where you can afford to generate many candidates and need a reliable way to select the best one.

What Comes Next

This chapter has expanded the scoring toolkit from Chapter 9's single scalar ORM to a richer set of options: process reward models that score each reasoning step, deterministic verifiers that check correctness without learned models, and combination strategies that use multiple scorers together.

The next three chapters put these scoring methods to work. Chapter 12 shows how GRPO combined with simple binary verifiers (math correctness, code execution) produced DeepSeek-R1's reasoning capabilities—a case where the reward signal was trivially simple but the training algorithm and scale made it powerful. Chapter 13 covers test-time compute, where best-of-N sampling, beam search, and Monte Carlo tree search all rely on the scoring methods from

this chapter to guide generation at inference time. Chapter 16 explores domain-specific applications where the choice of verifier or PRM depends on the task: execution feedback for code, formal provers for math, and conversation quality metrics for dialogue.

The scoring toolkit so far.

ORM (Chapter 9): One score per complete response. Simple, general-purpose. Best for tasks without clear step-by-step structure.

PRM (this chapter): One score per reasoning step. More informative for multi-step tasks. Expensive to train but produces dramatic improvements on reasoning benchmarks.

Verifier (this chapter): Deterministic correctness check. Free, perfect, unhackable—but only works for tasks with objective answers. The preferred reward signal for GRPO and RLOO (Chapter 10).

Ensemble (this chapter): Combine multiple scorers for best results. Use the two-stage pipeline (PRM filter → ORM/verifier select) when scoring cost matters.

The algorithms from Chapter 10 work with any of these scoring methods. DPO and KTO use implicit rewards from preferences. PPO and GRPO use explicit reward scores—from an ORM, PRM, verifier, or any combination. Matching the right scorer to the right algorithm and task is one of the most important practical decisions in the RL-for-LLMs pipeline.

Companion notebook. The code for this chapter is in `code/notebooks/part3_advanc` in the book repository at `github.com/arunpshankar/packt-final`. On GitHub, navigate to the file and replace `github.com` with `githubtocolab.com` in the URL to open it directly in Colab.